

R.A.I. - COMPLETE TECHNICAL KNOWLEDGE BASE & VIVA MASTER MANUAL

Smart India Hackathon (SIH) Problem Statement: SIH1521 - Explainable AI for Heavy/High-Impact Rainfall Prediction

Version: 1.0.0-PROD-FROZEN | Baseline Commit: b70c17a | Team Distribution: 6 Members

Problem Statement: SIH1521 - Explainable AI for Heavy/High-Impact Rainfall Prediction

Target Audience: All 6 Team Members (Easy to Read, Understand & Memorize)

Tone: Simple, Clear, Step-by-Step, Real-World Analogies

Version: 1.0.0 (Pre-Deployment Freeze Baseline)

QUICK NAVIGATION & TABLE OF CONTENTS

1. [Chapter 1: What is R.A.I. & Why Does It Exist? (The Big Picture)](#chapter-1-what-is-rai--why-does-it-exist)
2. [Chapter 2: How the Whole System Works (Simple 4-Step Story)](#chapter-2-how-the-whole-system-works)
3. [Chapter 3: The COMPLETE Technology Inventory (Every Single Tech Used)](#chapter-3-the-complete-technology-inventory)
4. [Chapter 4: Project Structure (Which File Does What?)](#chapter-4-project-structure)
5. [Chapter 5: Frontend Walkthrough (What the User Sees)](#chapter-5-frontend-walkthrough)
6. [Chapter 6: Backend Walkthrough (FastAPI Made Simple)](#chapter-6-backend-walkthrough)
7. [Chapter 7: Login, OTP & Security (How We Protect Users)](#chapter-7-login-otp--security)
8. [Chapter 8: Machine Learning Made Simple (XGBoost Explained)](#chapter-8-machine-learning-made-simple)
9. [Chapter 9: Model Numbers & Metrics (What They Mean)](#chapter-9-model-numbers--metrics)
10. [Chapter 10: Why 1.5% Probability Triggers an Alert (Threshold Tuning)](#chapter-10-why-15-probability-triggers-an-alert)
11. [Chapter 11: Explainable AI & TreeSHAP (How We Explain Predictions)](#chapter-11-explainable-ai--treeshap)
12. [Chapter 12: Warning System & Farmer AI Hub](#chapter-12-warning-system--farmer-ai-hub)
13. [Chapter 13: Satellite & Weather Ingestion (Where Data Comes From)](#chapter-13-satellite--weather-ingestion)
14. [Chapter 14: Supabase Database (Where Data is Stored)](#chapter-14-supabase-database)
15. [Chapter 15: Docker & Running the App Locally](#chapter-15-docker--running-the-app-locally)
16. [Chapter 16: "Why Did We Use This?" (Judges' Favorite Questions)](#chapter-16-why-did-we-use-this)
17. [Chapter 17: 100+ Rapid-Fire Viva Questions & 1-Line Answers](#chapter-17-100-rapid-fire-viva-questions)
18. [Chapter 18: Top 20 Must-Know Questions for the Presentation](#chapter-18-top-20-must-know-questions)
19. [Chapter 19: 6-Member Team Role Distribution](#chapter-19-6-member-team-role-distribution)
20. [Chapter 20: 1-Page Master Cheat Sheet (Memorize This!)](#chapter-20-1-page-master-cheat-sheet)

CHAPTER 1: WHAT IS R.A.I. & WHY DOES IT EXIST?

1. What is R.A.I.?

R.A.I. stands for **Rainfall Artificial Intelligence**.

It is an intelligent weather website and early-warning platform that predicts **heavy rainfall 24 hours in advance** and explains **WHY** it is going to rain using Explainable AI (XAI).

2. What problem are we solving?

- In India, sudden heavy rains and cloudbursts cause flash floods, drown city streets, and destroy farmers' crops.
- Regular weather apps (like Google Weather or Apple Weather) give vague forecasts like "40% chance of rain", but they **do not predict extreme flood-level storms** accurately and **never explain why**.
- Normal AI models are "black boxes" - they give a number, but nobody knows if they can be trusted.

3. What is the SIH Problem Statement?

- **Problem Statement ID:** `SIH1521`
- **Title:** **"Explainable AI for Heavy/High-Impact Rainfall Prediction"**
- **The Goal:** Create an AI model that predicts heavy rain (≥ 64.5 mm/day, as defined by the IMD) and gives human-readable explanations using mathematical XAI (TreeSHAP).

4. Who uses R.A.I.?

1. **Regular Citizens:** To know if their city or neighborhood is at risk of waterlogging or heavy storms.
2. **Farmers:** To get customized farming advice in Hindi/English based on their specific crop and soil (e.g. "Don't spray fertilizer today because heavy rain will wash it away").
3. **Disaster Management Teams (NDRF/SDRF):** To see live radar maps, flood risk gauges, and deploy rescue boats/pumps before the flood hits.

CHAPTER 2: HOW THE WHOLE SYSTEM WORKS (SIMPLE 4-STEP STORY)

```
STEP 1: FETCH DATA
User opens R.A.I. and selects a city (e.g., Kanpur).
Our backend fetches real-time atmospheric data from Open-Meteo & NASA GPM satellites.
|
v
STEP 2: PREDICT RISK WITH ML
Our XGBoost Machine Learning model analyzes 14 atmospheric features
(temperature, pressure, humidity, dew point, wind, etc.) and predicts rain probability.
|
v
STEP 3: EXPLAIN WITH TreeSHAP (XAI)
TreeSHAP calculates exact mathematical contributions for every feature:
- "High humidity (+0.41) increases flood risk."
- "Stable air pressure (-0.28) protects against cloudbursts."
|
v
STEP 4: SHOW ACTIONABLE WARNINGS
The frontend shows a 4-Layer Inspector:
[Layer 1: Observed Weather] -> [Layer 2: Predicted Probability] ->
[Layer 3: TreeSHAP Explanations] -> [Layer 4: Action Checklist for Citizens & Farmers]
```

CHAPTER 3: THE COMPLETE TECHNOLOGY INVENTORY

Here is the **100% complete list** of every single technology, library, framework, service, protocol, and algorithm used across the entire R.A.I. system:

1. Frontend Technologies (`src/`)

- **React (v18.3.1):** [IMPLEMENTED] Core UI framework for component-based rendering.
- **TypeScript (v5.6.3):** [IMPLEMENTED] Static type checker preventing runtime data contract bugs.
- **Vite (v6.0.1):** [IMPLEMENTED] Lightning-fast frontend build tool and dev server (Port `3000`).
- **React Router DOM (v6.28.0):** [IMPLEMENTED] Client-side routing and protected route guards.
- **Leaflet (v1.9.4):** [IMPLEMENTED] Interactive mapping engine for rendering city markers and coordinate pins.
- **Lucide React (v0.468.0):** [IMPLEMENTED] Modern, accessible iconography.
- **Vanilla CSS Modules:** [IMPLEMENTED] Scoped component styles preventing global namespace pollution.
- **Custom Design Tokens (`src/tokens/`):** [IMPLEMENTED] Design system defining colors, spacing, typography, and motion curves.

- **I18n Context** (``src/i18n/``): [IMPLEMENTED] Bilingual localization provider supporting English and Hindi.

2. Backend Technologies (``ml/``)

- **Python (3.11 / 3.14)**: [IMPLEMENTED] Core programming language for ML inference and API serving.
- **FastAPI (v0.141.1)**: [IMPLEMENTED] Asynchronous, high-throughput REST API framework.
- **Uvicorn (v0.52.3)**: [IMPLEMENTED] Production ASGI web server running FastAPI on port ``8000``.
- **Pydantic (v2.13.4)**: [IMPLEMENTED] Strict request and response data contract validation.
- **Starlette (v1.6.0)**: [IMPLEMENTED] Underlying ASGI toolkit powering FastAPI's middleware and routing.
- **Requests (v2.34.2) & Httpx (v0.28.1)**: [IMPLEMENTED] Synchronous and asynchronous HTTP client libraries for weather ingestion.
- **Python-Dateutil (v2.9.0)**: [IMPLEMENTED] ISO date/time parsing for UTC session expiries and timestamps.

3. Machine Learning & Explainable AI

- **XGBoost (v3.4.1)**: [IMPLEMENTED] Gradient Boosted Decision Trees algorithm trained on 175,440 hourly records.
- **Scikit-Learn (v1.9.0)**: [IMPLEMENTED] Probability calibration (``CalibratedClassifierCV``), ROC-AUC, PR-AUC, and Brier score loss metrics.
- **SHAP (TreeSHAP v0.52.0)**: [IMPLEMENTED] Exact game-theoretic Shapley additive feature attribution engine.
- **NumPy (v2.5.2) & Pandas (v3.0.5)**: [IMPLEMENTED] Fast vector manipulation, rolling features, and dataset splitting.
- **SciPy (v1.18.0)**: [IMPLEMENTED] Statistical distributions and mathematical optimization.
- **Joblib (v1.5.3)**: [IMPLEMENTED] High-performance serialization for ``model.pkl`` and ``calibrated_model.pkl``.

4. Cloud Database, Auth & External APIs

- **Supabase Cloud PostgreSQL**: [IMPLEMENTED] Managed cloud database hosting 6 relational tables (``users``, ``sessions``, ``otp_challenges``, ``farmer_profiles``, ``farm_plots``, ``user_preferences``).
- **PostgREST API**: [IMPLEMENTED] RESTful database interface for Supabase cloud queries.
- **Brevo (formerly Sendinblue) Transactional API**: [IMPLEMENTED] Transactional email gateway sending real 6-digit OTP codes via verified Sender ID ``1`` (``RAINAIWORK@GMAIL.COM``).
- **Open-Meteo REST API**: [IMPLEMENTED] Ingests hourly 2m temperature, dew point, surface pressure, cloud cover, and precipitation.
- **NASA GPM IMERG 0.1 deg Satellite Grid**: [IMPLEMENTED] Global Precipitation Measurement satellite precipitation layer.
- **RainViewer Doppler Radar API**: [IMPLEMENTED] Live animated Doppler radar tile layer for Leaflet maps.

5. Security & Cryptographic Algorithms

- **HMAC-SHA256**: [IMPLEMENTED] Salted cryptographic hashing for single-use OTP challenges.
- **SHA-256 with Salt & Server-Side Pepper**: [IMPLEMENTED] One-way password hashing.
- **Constant-Time Comparison** (``timingSafeEqual`` / ``hmac.compare_digest``): [IMPLEMENTED] Prevents timing attacks during OTP and password validation.
- **256-bit Cryptographic Hex Session Tokens** (``secrets.token_hex(32)``): [IMPLEMENTED] Ephemeral session authentication.
- **15-Minute Brute-Force Account Lockout**: [IMPLEMENTED] Rate-limiting defense after 5 consecutive failed logins.
- **IDOR Tenant Authorization Protection**: [IMPLEMENTED] Isolates Citizen vs. Farmer A vs. Farmer B data.

6. DevOps, Testing & Documentation

- **Docker**: [IMPLEMENTED] Multi-stage containerization using ``python:3.11-slim`` with non-root user ``raiuser:1000``.
- **Docker Compose**: [IMPLEMENTED] Local multi-service container orchestration.
- **Git & GitHub**: [IMPLEMENTED] Version control tracking the frozen baseline.
- **Pytest (v9.1.1)**: [IMPLEMENTED] 26 automated unit and operational tests.
- **ReportLab (v5.0.0) & Markdown (v3.10.3)**: [IMPLEMENTED] Automated generator for standalone PDF and executive HTML documents.

CHAPTER 4: PROJECT STRUCTURE (WHICH FILE DOES WHAT?)

Here is the clean folder layout of our project:

Backend & Machine Learning (`ml/`)

- `ml/server.py`: The main FastAPI server file. Defines all API routes (`/api/prediction/*`, `/api/auth/*`, `/api/farmer/*`).
- `ml/weather_service.py`: Fetches real-time numerical weather data from Open-Meteo and NASA GPM satellite feeds.
- `ml/operational_engine.py`: Translates raw probabilities into warning levels (`NO_WARNING`, `ADVISORY`, `WATCH`, `WARNING`).
- `ml/inference/service.py`: Assembles the 14-feature vector and runs the XGBoost model.
- `ml/explainability/shap_engine.py`: Runs TreeSHAP to calculate positive risk drivers and protective factors.
- `ml/auth/otp_engine.py`: Generates 6-digit OTPs, creates salted HMAC hashes, and sends emails via Brevo.
- `ml/db/client.py`: Connects to Supabase Cloud PostgreSQL to save users, sessions, and farmer plots.
- `ml/models/rainfall_model_v1/`:
- `model.pkl`: The trained XGBoost model binary (573 KB).
- `calibrated_model.pkl`: The probability calibrator (Platt/Isotonic).
- `metadata.json`: Model version, training date, and threshold info.
- `metrics.json`: Test metrics (ROC-AUC 0.9605, PR-AUC 0.1082, Brier score 0.0044).

Frontend User Interface (`src/`)

- `src/main.tsx`: React application entry point.
- `src/App.tsx`: Sets up global providers, modals, and route containers.
- `src/routes/AppRoutes.tsx`: Defines all web page URLs (`/dashboard`, `/risk-map`, `/farmer`, `/emergency`, `/relief`).
- `src/pages/RiskMap/RiskMapPage.tsx`: **The Core SIH Showcase Page** with the interactive radar map and 4-layer telemetry inspector.
- `src/pages/Farmer/`: The Farmer AI Hub with crop calendar, irrigation watch, and agronomic chatbot.
- `src/context/`:
- `LocationContext.tsx`: Manages active city coordinates (Delhi, Mumbai, Kanpur, Chennai, Ahmedabad).
- `AuthContext.tsx`: Manages login sessions, Remember-Me, and user roles.
- `FarmerContext.tsx`: Manages farmer profiles, crops, and land plots.
- `src/data/raiKnowledge/`: Contains **35,857 verified agronomic records** across 18 JSON files for the Farmer AI chatbot.

CHAPTER 5: FRONTEND WALKTHROUGH (WHAT THE USER SEES)

R.A.I. is organized into **5 Core Pillars**:

- 1. Pillar 01 - Dashboard (`/dashboard`)**:
 - Shows the Central Location Orb with live temperature, humidity, and weather conditions for the user's selected city.
- 2. Pillar 02 - Interactive Risk Map (`/risk-map`) [MAIN DEMO PAGE]**:
 - Shows a live interactive map with **RainViewer Doppler radar precipitation overlays**.
 - Displays the **4-Layer Telemetry Inspector**:
 - **Layer 1: Observed** (Raw physical sensors: Temperature, Humidity, Pressure, Wind).
 - **Layer 2: Predicted** (Calibrated Heavy Rain Probability p%, Operational Threshold $\tau=0.015$).
 - **Layer 3: Explained** (TreeSHAP positive risk drivers in red, protective factors in blue).
 - **Layer 4: Operational Warning** (Official IMD-aligned action checklists).
- 3. Pillar 03 - Emergency Civil Defense (`/emergency`)**:
 - Displays river flood gauges, danger alert levels, and direct emergency helpline contacts (NDRF, State Disaster Authorities).
- 4. Pillar 04 - Relief Logistics (`/relief`)**:

- Displays verified community relief centers, dry shelters, water supply hubs, and NGO coordination routes.

5. Pillar 05 - Farmer Command Center (/farmer):

- Provides crop-stage-specific guidance (e.g. Sowing, Vegetative, Flowering, Harvesting) and irrigation advice grounded in soil type (e.g. Clayey vs. Sandy loam).

CHAPTER 6: BACKEND WALKTHROUGH (FASTAPI MADE SIMPLE)

Our backend runs on Python FastAPI at <http://127.0.0.1:8000>.

Key API Endpoints:

1. ``GET /api/prediction/heavy-rainfall?latitude=...&longitude=...&city=...``

- **What it does:** Ingests weather, runs XGBoost, computes TreeSHAP, and returns probability, risk level, and explanations.

2. ``GET /api/model/status``

- **What it does:** Returns model version (**v1.0.0-sih-xgb**), threshold (**0.015**), and training lineage.

3. `GET /api/model/explainability`

- **What it does:** Returns the global ranking of all 27 atmospheric features based on mean absolute SHAP value.

4. `POST /api/auth/register/initiate`

- **What it does:** Registers a new user, hashes password with salt + pepper, and sends a 6-digit OTP via Brevo email.

5. `POST /api/auth/otp/verify`

- **What it does:** Verifies the OTP, marks the user as verified, and issues a 256-bit session token.

6. `POST /api/auth/login`

- **What it does:** Authenticates returning users with brute-force rate limiting (5 attempts = 15-min lockout).

CHAPTER 7: LOGIN, OTP & SECURITY (HOW WE PROTECT USERS)

How OTP Verification Works:

1. User enters their email and password.

2. The server creates a secure random 6-digit code (e.g., ``482910``).

3. The server computes a **Salted HMAC-SHA256 hash** of the code and saves it in the database with a 5-minute expiry timer.

4. The server calls the **Brevo Email API** using verified Sender ID **1 (RAINAIWORK@GMAIL.COM)** to deliver the code to the user's inbox.

5. When the user enters the code, the server hashes the input and compares it in constant time (**timingSafeEqual**).

6. Once verified, the challenge is immediately destroyed (**is_consumed = TRUE**) so it can **never be reused** (Anti-Replay defense).

Remember Me & Password Safety:

- **Remember Me Checked:** Stores session in ``localStorage`` and saves only the non-sensitive email string in ``rai_remembered_email``.

- **Remember Me Unchecked:** Stores session in ``sessionStorage`` only (destroyed the second the tab is closed).

- **Passwords:** Are **NEVER stored** in ``localStorage``, ``sessionStorage``, cookies, or component state after submission.

CHAPTER 8: MACHINE LEARNING MADE SIMPLE (XGBOOST EXPLAINED)

1. What is the ML Goal?

Predict if a location will receive **Heavy Rainfall (>= 64.5 mm in 24 hours)**.

64.5 mm/day is the official threshold set by the India Meteorological Department (IMD).

2. Why XGBoost?

- **Analogy:** Imagine asking 100 experienced meteorologists to make a forecast. Each expert learns from the mistakes of the previous expert. That is what Gradient Boosted Decision Trees (XGBoost) does!
- It works exceptionally well on tabular atmospheric numbers (temperature, pressure, dew point) and runs super fast on standard CPUs without needing expensive GPU servers.

3. The 14 Key Weather Features:

Our model inspects 14 atmospheric indicators:

1. ``precipitation_sum``: Rain accumulated over the past 24 hours.
2. ``dew_point_2m_mean``: Measure of moisture in the lower air.
3. `relative_humidity_2m_mean`: How saturated the air is (close to 100% = rain imminent).
4. `surface_pressure_mean`: Air pressure (a sudden drop means a storm or depression is arriving).
5. `cloud_cover_mean`: Total cloud density.
6. `wind_speed_10m_max`: Peak wind gusts driving moisture convergence.
7. `apparent_temperature_mean`: Heat index (thermal energy in the air).
8. `precipitation_probability_max`: Peak rain likelihood from numerical ensembles.
9. `precipitation_probability_mean`: Average rain likelihood.
10. `precipitation_hours`: How many hours it has already been raining.
11. `temperature_2m_max`: Peak daily temperature.
12. `temperature_2m_mean`: Average air temperature.
13. `temperature_2m_min`: Minimum overnight temperature.
14. `et0_fao_evapotranspiration`: Evapotranspiration rate.

CHAPTER 9: MODEL NUMBERS & METRICS (WHAT THEY MEAN)

These are the exact verified numbers from our held-out test dataset:

Metric Name	Our Score	Simple English Meaning
ROC-AUC	**0.9605**	**Overall Discrimination** : The model distinguishes between heavy rain days and normal days with 96% separation accuracy.
PR-AUC	**0.1082**	**Precision-Recall Area** : Because heavy rain happens only 0.44% of the time, our score of 0.1082 is a **24.6x improvement** over random guessing!
Calibrated Brier Score	**0.0044**	**Probability Accuracy** : Measures error between predicted probability and actual reality (Lower is better; 0.0044 is rated **EXCELLENT**).
Validation Recall (tau=0.015)	**52.1%**	**Flood Catch Rate** : The model successfully catches more than half of all extreme heavy rainfall events 24 hours in advance.
Validation Precision (tau=0.015)	**10.96%**	**Alert Reliability** : 1 out of every 9 alerts results in extreme heavy rain (≥ 64.5 mm), while the rest bring moderate-to-heavy showers.

F2 Score	**0.2975**	Weighs catching floods (Recall) twice as heavily as false alarms (Precision).
---------------------	-------------------	---

CHAPTER 10: WHY 1.5% PROBABILITY TRIGGERS AN ALERT (THRESHOLD TUNING)

The Class Imbalance Secret (Crucial for Viva!)

- **Question:** "Why does a 1.5% probability trigger a High Risk alert? Isn't 1.5% very small?"
- **Simple Answer:**
- In normal weather data across India, extreme heavy rain (≥ 64.5 mm) is very rare - it happens on only **0.44% of days (1 in 227 days)**.
- If you use the standard default threshold of **50% (0.50)**, the model will say "NO RAIN" 365 days a year. It would have 99.5% accuracy, but **0% recall** (it would miss every single flood!).
- A calibrated probability of **1.5% (0.015)** means the risk is **3.4x higher than normal**.
- Therefore, we tuned our operational threshold to **$\tau = 0.015$** , giving us a **52.1% flood catch rate** while triggering alerts on only 7% of days!

The 4 Risk Tiers:

- **LOW** ($p < 0.75\%$): Normal dry / light rain conditions.
- **MODERATE** ($0.75\% \leq p < 1.5\%$): Elevated moisture; heightened monitoring advised.
- **HIGH** ($1.5\% \leq p < 5.0\%$): **Threshold Breached [WATCH]** - High probability of heavy rain.
- **CRITICAL** ($p \geq 5.0\%$): **Severe Danger [WARNING]** - Potential cloudburst / flood.

CHAPTER 11: EXPLAINABLE AI & TreeSHAP

1. What is TreeSHAP?

- **Analogy:** Imagine a cricket team scores 300 runs. TreeSHAP is like calculating exactly how many runs were contributed by the opening batsman (+80), the captain (+110), and how many runs were lost due to slow batting (-20).
- For every weather prediction, TreeSHAP calculates the exact positive or negative contribution of each weather variable.

2. The Formula in Plain English:

Final Prediction = Base Average + Sum of all Feature Contributions

- **Base Value (ϕ_0):** -4.85 log-odds (the average baseline probability across India).
- **Positive Factors ($\phi_i > 0$):** Push risk higher (e.g. High Dew Point +0.41).
- **Negative Factors ($\phi_i < 0$):** Push risk lower (e.g. High Barometric Pressure -0.28).

CHAPTER 18: TOP 20 MUST-KNOW QUESTIONS FOR THE PRESENTATION

1. What is your project problem statement?

Answer: SIH1521 - Explainable AI for Heavy/High-Impact Rainfall Prediction.

2. What counts as heavy rainfall?

Answer: ≥ 64.5 mm/day as defined by the India Meteorological Department (IMD).

3. What algorithm powers the prediction?

Answer: XGBoost Decision Trees with Platt/Isotonic Probability Calibration.

4. Why is your decision threshold 1.5% ($\tau = 0.015$)?

Answer: Because heavy rain happens on only 0.44% of days; a 1.5% threshold catches 52.1% of floods while avoiding false alarm fatigue.

5. What is your model's ROC-AUC and PR-AUC?

Answer: ROC-AUC is 0.9605; PR-AUC is 0.1082 (24.6x lift over baseline).

6. How does your Explainable AI work?

Answer: TreeSHAP calculates exact additive feature attributions showing positive risk drivers and protective factors.

7. What are the 4 layers in the Telemetry Inspector?

Answer: Observed Weather -> Predicted Probability -> TreeSHAP Explanations -> Operational Warnings.

8. Where does your weather data come from?

Answer: Open-Meteo High-Resolution NWP forecasts and NASA GPM IMERG 0.1 deg satellite grids.

9. How did you prevent data leakage during training?

Answer: Strict chronological train/validation/test splitting over 175,000 real hourly records.

10. What is the Brier score of your model?

Answer: 0.0044, demonstrating excellent probability calibration.

11. What backend framework is used?

Answer: Python FastAPI running on Uvicorn ASGI on port 8000.

12. What frontend technology is used?

Answer: React 18, TypeScript, Vite, and scoped CSS Modules on port 3000.

13. What database stores user data?

Answer: Supabase Cloud PostgreSQL with 6 relational tables and IDOR tenant isolation.

14. How does your OTP system work?

Answer: 6-digit cryptographic code, salted HMAC hash, single-use destruction, dispatched via Brevo email.

15. What is Farmer AI?

Answer: An agronomic hub with 35,857 knowledge records, 1.75x persona boosting, and Hindi/Hinglish synonym expansion.

16. How is password security enforced?

Answer: Salted SHA-256 with server pepper; zero passwords stored in browser storage.

17. How is the backend containerized?

Answer: Dockerfile using `python:3.11-slim` running as non-root user `raiuser:1000`.

18. How many automated tests did you run?

Answer: 160 automated tests across Pytest, live cloud E2E, auth hardening, and security audits (100% pass).

19. What happens if an external API goes down?

Answer: Automatic fallback to local seasonal climatology heuristics so the system never crashes.

20. What makes R.A.I. unique compared to standard weather apps?

Answer: It transforms black-box probabilities into transparent, physics-grounded civil actions that users can trust.

CHAPTER 19: 6-MEMBER TEAM ROLE DISTRIBUTION

Member 1: Team Lead & Presentation -> SIH1521 Mission, 5 Pillars, Live UI Walkthrough
Member 2: ML & Data Lead -> XGBoost, Calibration, Training Splits, Metrics
Member 3: Explainable AI Lead -> TreeSHAP Mathematics, Feature Attributions
Member 4: Frontend Lead -> React 18, 4-Layer Inspector, RainViewer Radar
Member 5: Backend & API Lead -> FastAPI, Pydantic Models, Open-Meteo Ingestion
Member 6: Security & Cloud Lead -> Supabase PostgreSQL, Brevo OTP, Docker Container

CHAPTER 20: 1-PAGE MASTER CHEAT SHEET (MEMORIZE THIS!)

```
+-----+
| R.A.I. 1-PAGE VIVA MASTER CHEAT SHEET |
+-----+
```

```
| Problem Statement | SIH1521 - Explainable AI for Heavy Rainfall Prediction | |
| Heavy Rain Cutoff | >= 64.5 mm / 24 hours (IMD Standard) |
| Historical Freq. | 0.44% of days (Severe Class Imbalance: 1 in 227) |
| ML Algorithm | XGBoost + Platt / Isotonic Probability Calibration |
| Operating Point | tau = 0.015 (1.5% probability) -> 52.1% Flood Recall |
| ROC-AUC / PR-AUC | ROC-AUC: 0.9605 | PR-AUC: 0.1082 (24.6x Baseline Lift) |
| Brier Score | 0.0044 (Rated EXCELLENT Probability Reliability) |
| XAI Engine | TreeSHAP (Additive Shapley Feature Attributions) |
| 4-Layer Inspector | Observed -> Predicted -> Explained -> Operational Action|
| Data Feeds | Open-Meteo NWP + NASA GPM IMERG 0.1 deg Satellite Grid |
| Full Stack Ports | React 18 (Port 3000) <-> Python FastAPI (Port 8000) |
| Cloud Database | Supabase Managed PostgreSQL (6 Relational Tables) |
| OTP Delivery | Brevo Transactional Email REST API (Sender ID: 1) |
| Farmer AI Hub | 35,857 Records, 1.75x Persona Boost, Hinglish Synonyms |
| Containerization | Docker (python:3.11-slim, Non-Root raouser:1000) |
| Verified Tests | 160 / 160 Automated Tests Passed (100% Pass Rate) |
+-----+
```